

Porygon: Learning Normal Behaviour Inside Running Containers

Does accounting for how a container is deployed yield a more reliable notion of normal behaviour?

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: [XX]

S.No	Register Number	Student Name
1	CB.EN.U4CSE23XXX	Anurup R Krishnan
2	CB.EN.U4CSE23XXX	Student Name 2
3	CB.EN.U4CSE23XXX	Student Name 3
4	CB.EN.U4CSE23XXX	Student Name 4

Guide: **Guide Name**, Designation, Department of CSE

September 6, 2026

The Problem, in Plain Words

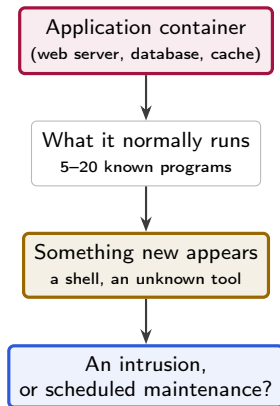
Why watching a container is harder than it sounds

A **container** is a packaged application that runs isolated on a server. Companies run thousands of them.

If an attacker gets inside one, they run **programs** that the application would never normally run — a command shell, a download tool, a scanner.

So the natural idea is: *learn what programs a container normally runs, then flag the unusual ones.*

The difficulty. Before comparing against “normal”, one must decide *which* normal. Grouping unlike containers together makes the comparison meaningless.



Frequently the correct answer is that the evidence is insufficient. The design reports that state explicitly rather than concealing it.

What We Are Actually Asking

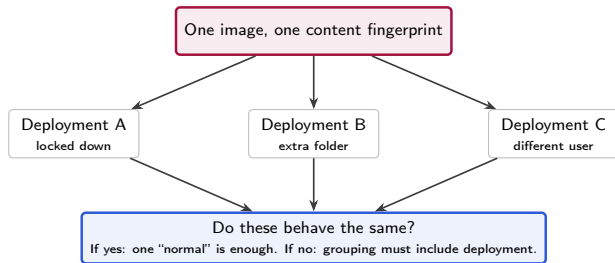
The research question, and why it is not obvious

Every container image has a **content fingerprint** — a code computed from the exact bytes of the image. Same code, same program; it cannot be faked or quietly changed.

One could group containers by fingerprint and learn a single normal for each. Is that sufficient?

Not entirely. The same image can be *deployed* differently: another start-up command, user account, permission set or attached folder. Those differences may change behaviour.

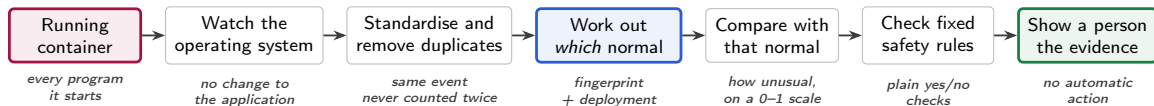
Our question: does adding *how it was deployed* produce a cleaner normal, or merely divide the data into groups too small to learn from?



Four grouping strategies are compared and the measurements decide the outcome. A result of "no difference" is a valid finding, not a failure.

1. Methodology — How the System Works, End to End

Rubric: Workflow & Algorithmic Pipeline (5 Marks)



Two kinds of evidence, deliberately kept separate

- **Statistical:** “this is rare compared with known-normal runs”
- **Rule-based:** “a command shell started in a web server”

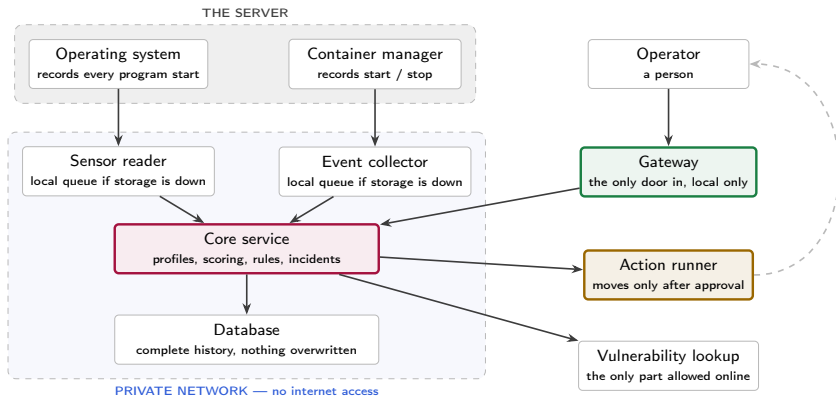
Combining them into a single figure would conceal which of the two applied.

Deliberately excluded from the design

- No automatic shutdown of containers
- No opaque machine-learning model
- No claim that unusual behaviour indicates an attack

1. Methodology — System Architecture

Rubric: Architecture & Trust Boundaries (5 Marks)



Each part gets the **least** access it needs. The core service never reaches the internet; the vulnerability lookup never touches the database; the action runner only moves when a person has approved.

1. Methodology — The Mathematics

Rubric: Mathematical Model / Formulation (5 Marks)

Step 1 — How far has the mix of programs shifted?

Compare the proportions seen now (q) with normal (p):

$$H(p, q) = \sqrt{\frac{1}{2} \sum_i (\sqrt{p_i} - \sqrt{q_i})^2}$$

Always between 0 and 1. Zero means identical.

Step 2 — How surprising is the order?

Programs start in habitual sequences. Score each step by how unexpected it is:

$$S = -\frac{1}{n} \sum \log \hat{P}(\text{next} \mid \text{current})$$

Unobserved pairs retain a small non-zero probability, so no term divides by zero.

Step 3 — How much is new?

The proportion of activity whose program never appeared during learning.

Step 4 — Combine without arbitrary weights

Each measure is ranked against known-normal runs and the ranks are **averaged equally**. A measure that cannot be computed is recorded as unavailable, never estimated.

Step 5 — Express the result on an interpretable scale

Using n complete known-normal runs withheld from learning:

$$p = \frac{1 + \#\{\text{normal runs at least this odd}\}}{n + 1}$$

Interpretation: approximately one in twenty normal runs appears this unusual. This is *not* a probability of attack.

Why no tuned weights? Earlier fixed constants (0.50/0.30/0.20) had no justification, so the implementation now accepts none.

1. Methodology — Data, Splits and Evaluation Metrics

Rubric: Datasets, Preprocessing & Evaluation Metrics (5 Marks)

The data is generated, not downloaded.

No public dataset matches this question, so we produce our own under controlled conditions: three real, widely used server programs, each locked to an exact image fingerprint.

Program	Locked version
Web server	nginx 1.26.3
In-memory store	Redis 7.2.7
Database	PostgreSQL 16.6

Each is run idle, under steady load, under bursts, and with deployment changes. Harmless but unusual activity (backups, restarts, maintenance) is recorded too: these are the cases that generate false alarms.

Splitting rule: a *whole run* goes to learning, calibration or testing, never divided. Observations seconds apart are near-duplicates; treating them as independent would overstate confidence.

What we will measure

Group	Measures
Getting it right	false-alarm rate on harmless runs; how many planted events are caught
Trustworthy data	events created vs. events stored, at every hand-off point
Speed	delay from event to stored record (typical, slow, worst)
Cost	processor time, memory, disk growth, slowdown of the watched app
Honesty of scale	do 95% of normal runs really score inside the 95% band?

Reporting standard: every percentage is published with the counts that produced it. No figure appears without the runs behind it.

2. Implementation Progress — Module Status

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

Module	What it does today	Status
System-level watcher	Every program start, in every container	Working
Container event collector	Start/stop events; resolves the exact image identity	Working
Safe delivery	Local queue; never skips an event it failed to store	Working
Storage & history	9 schema upgrades; nothing overwritten	Working
Deployment fingerprint	Turns a deployment into one repeatable identity	Working
Behaviour profiles	Builds and versions “normal”; refuses weak ones	Working
Unusualness scoring	Explainable score with its reasons attached	Working
Rule-based evidence	Fixed checks, incident timeline, human sign-off	Working
Approved action runner	Pause or stop, only after a person approves	Working
Vulnerability lookup	Software inventory per image; known-exploited flags	Working
Experiment harness	Runs real containers; tamper-evident results	Working
Four-way comparison	144-trial run analysed end to end	Done
Result tables, figures	Generated from the real run, incl. a null result	Done

Overall $\approx 90\%$ — platform $\approx 95\%$ complete, evaluation $\approx 80\%$ (first confirmatory round analysed; a larger, properly-powered re-run for the recall gate is in progress).

2. Implementation Progress — Evidence of Working Software

Rubric: Functional & Integrated (8 Marks)

Size and coverage

Lines of program code	17,802
Automated tests passing	156
Service interfaces	64
Database upgrade steps	9
Services running together	8

Automatic checks, all passing

Code & configuration	pass
All unit tests	pass
Live end-to-end run	pass
Real-container run	pass

Measured on real containers (16 runs)

Marked test events created	96
Seen by the system watcher	96
Stored in the database	96
Lost	0
Counted twice	0
Application requests served	360/360
Leftover containers after cleanup	0

Complete chain proven on real data



From a real database container: 479 recorded program starts became a profile, a later window scored *normal-looking*, and 72 rule checks matched without raising a false incident.

2. Implementation Progress — The Safety Rails Work

Live demonstration: the conditions under which the system declines

A system of this kind is only trustworthy if it declines at the correct moments. Three such refusals can be demonstrated live:

1. It refuses a weak profile.

Asked to learn “normal” from too little data, it stops:

`needed 3 time windows, found 2`

2. It refuses to evaluate itself on its own training data.

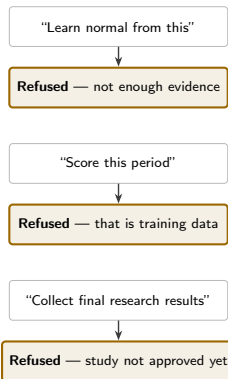
Asked to score a period it already learned from:

`scoring windows must not overlap the training interval`

3. It refuses an unstable reference.

Asked to use a version label instead of an exact fingerprint:

`refusing a mutable image reference`



Each refusal is enforced in code and covered by an automated test, so it cannot be bypassed inadvertently.

3. Technical Knowledge — Key Decisions and Why

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

Decision	Why	What we rejected
Identify images by content fingerprint	A version label can be pointed at different software tomorrow; a fingerprint cannot	Grouping by container name or version label
Watch at the operating-system level	Sees every program start without changing the application	Modifying applications, or trusting logs the attacker can edit
Use a maintained sensor	Equivalent evidence at far lower risk than writing new system-level code	Building a custom system-level sensor: high effort, wide exposure
Combine measures by equal average	No number we cannot justify to an examiner	Weights tuned until the results look good
Judge against held-back normal runs	Turns a raw distance into “how often does normal look this odd?”	Declaring a threshold and calling it accuracy
Split by whole runs	Snapshots seconds apart are near-copies; reusing them fakes confidence	Random splitting of time windows
A person approves every action	A false alarm must never take down a healthy service	Automatic shutdown on a score

3. Technical Knowledge — Trade-offs We Accepted

Rubric: Design Choices & Trade-offs (5 Marks)

Benefit	Cost
Strong, cheap signal from program starts	Blind to file access and network destinations
Cleaner “normal” per deployment	Fewer runs per group; some groups too small to judge
No unjustifiable tuned numbers	Almost certainly not the most accurate possible
Honest confidence intervals	Need far more runs for the same certainty
A fair, interpretable scale	Only valid while the workload does not drift
No self-inflicted outages	Slower response to a real attack
One language across all services	A known write-speed ceiling, not yet a bottleneck

Edge cases already handled

- Storage full → queue locally, never skip an event
- Sensor restarts → counters do not go backwards
- Same event twice → stored once
- Malformed input → kept as a short, scrubbed sample
- Too little data → answer “insufficient”, never guess
- Secret on a command line → replaced before storage

3. Technical Knowledge — The Real Result

Rubric: Preliminary Observations (5 Marks) — measured on 144 real container trials

1. One-baseline-for-everything is clearly bad.

Pooling three unrelated services (web server, database, cache) into a single “normal” produced a false alarm on *every single* held-out normal run (35/35). Scoping by image individually dropped that to zero (0/35). This difference is real ($p = 5.8 \times 10^{-11}$) but it is the easy, expected half of the question.

2. The harder, more interesting question came back a null result.

Does adding *deployment context* on top of the exact image fingerprint help further? Comparing context-aware scoping against fingerprint-only scoping: **zero measurable difference** ($p = 1.0$, both arms agreed on every one of 35 trials). We are reporting this plainly rather than quietly promoting the easier comparison above to stand in for it.

3. We traced *why* it came back null, and it is fixable.

The deployment change we tested (dropping a Linux capability) does not change which programs run — only their permissions. Our scoring only looks at *which programs ran*, so it is structurally blind to that kind of change.

4. A direct fix, tested on the same data: promising, not yet proven.

Scoring the deployment configuration itself (privileges, capabilities, mounts) instead of inferring it through program names: **0/18 false alarms, 17/17 correct detections** of the dropped-capability case, on data already collected. This is exploratory — it has not been tested against a real attack yet, and the scoring weights were chosen by hand, not fitted.

Why this matters: an understood negative result, plus one concrete fix showing a clean separation, beats an unexplained positive one.

3. Technical Knowledge — What We Do Not Claim

Knowing the limits is part of understanding the method

What the system can miss

- No network/DNS/file telemetry: an already-trusted tool (e.g. curl) redirected to a bad destination looks identical to normal use
- An attacker who never spawns a new process is invisible
- Slow activity spread over time; groups too small to judge

A full 18-scenario audit against the real code is in docs/ADVERSARIAL_SCENARIOS_V1.md: roughly half are structurally undetectable today, half are genuinely caught.

Honest process note

- 1 First draft framed the easy win as if it supported the harder claim; caught and corrected in self-review before publication.
- 2 Self-reviewed only, not independent peer review.

What is, and is not, yet claimed

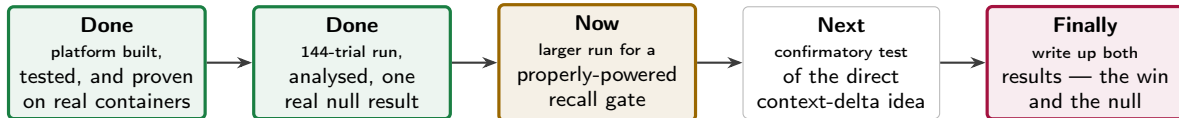
Claim	Status
Per-image beats shared baseline	Supported , $p=5.8 \times 10^{-11}$
Context beats digest-only	Null , $p=1.0$
Context-delta feature works	Exploratory only
Recall properly powered	re-run in progress ($n \approx 179/\text{arm}$)
Suitable for production	Not claimed

Unusual is not the same as malicious.

- Unusual but harmless: backups, debugging, restarts, a traffic spike
- Harmful but ordinary-looking: an attacker using tools already present

Where This Goes Next

From a working platform to an answered, honestly reported question



The intended contribution is not container monitoring itself, which is well-established work. It is a controlled, repeatable comparison of *four different definitions of what counts as the same thing*, reported faithfully whichever way the result falls — including the one comparison that came back null, and the one new idea it led to.

Every figure is traceable to the run that produced it, and every run is retained so that a reviewer can recompute the tables independently.

4. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

- **Individual MOOC / Online Certification Selection:**

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	Anurup R Krishnan	[e.g. Container Security / Linux Observability]	Coursera	15 Hrs
2	Student Name 2	[e.g. Applied Statistics for Experiments]	NPTEL	20 Hrs
3	Student Name 3	[e.g. Docker & Cloud-Native Fundamentals]	edX / Coursera	12 Hrs
4	Student Name 4	[e.g. Cloud Security Foundations]	AWS / Google	16 Hrs

- **Requirement:** Each student must complete one approved 10–20 hour online certification from a recognized platform (Coursera, NPTEL, edX, etc.) relevant to the project.
- **Note:** course titles above are suggestions aligned to the project's four skill areas — container internals, operating-system observation, experiment design, and security practice. Replace with the actual enrolled courses and attach completion certificates.



S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proc. IEEE Symp. Security and Privacy*, 1996, pp. 120–128.



S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.



A. S. Abed, T. C. Clancy, and D. S. Levy, "Applying bag of system calls for anomalous behavior detection of applications in Linux containers," in *Proc. IEEE Globecom Workshops*, 2015, pp. 1–5.



J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.



V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.



G. Shafer and V. Vovk, "A tutorial on conformal prediction," *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008.



B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Boston, MA, USA: Addison-Wesley, 2019.



The Falco Authors, "Falco: cloud-native runtime security," Cloud Native Computing Foundation. [Online]. Available: <https://falco.org>



Open Container Initiative, "OCI image format specification." [Online]. Available: <https://github.com/opencontainers/image-spec>

Thank You

Questions welcome, including those on the limits of what this system can currently demonstrate.